

Pseudojęzyk

1. Podstawy

Pseudokodem nazywany jest taki sposób zapisu algorytmu, który zachowując strukturę charakterystyczną dla kodu zapisanego w języku programowania, rezygnuje ze ścisłych reguł składniowych na rzecz prostoty i czytelności.

Aby zwiększyć czytelność algorytmu, przyjęło się, że instrukcje sterujące są zapisywane **WIELKIMI LITERAMI**.

Każdy algorytm musi posiadać miejsce rozpoczęcia.

Instrukcja startu powinna wskazywać na fakt rozpoczęcia poprzez zapis: *START, Rozpocznij, Zacznij*, itp.

Instrukcja rozpoczęcia może być tylko jedna w całym algorytmie. Każdy algorytm musi posiadać instrukcję końca.

Instrukcja końca powinna wskazywać na fakt zakończenia poprzez zapis: *STOP, koniec, Zakończ*, itp.

Czasami dla zwiększenia czytelności kodu stosuje się kilka instrukcji zakończenia, gdy wystąpi określona sytuacja i algorytm należy przerwać. Lepsze to (pod względem czytelności zapisu) niż wykonywanie skoków ze środka algorytmu do końca.

Jeśli używamy programu (notatnik, word, writer, itp.) do zapisu kodu w pseudojęzyku, to powinniśmy ustawić typ czcionki na jakiegoś „kuriera” np. „courier”, „consolas”, „lucida console”, itp.. Są to czcionki, gdzie wszystkie znaki są tak samo szerokie (niezależnie czy to jest litera „i” czy „w”) przez co znaki ustawiają się ładnie „jedne nad drugimi”, zwiększona jest czytelność kodu, łatwiej sterować wcięciami.

Czasami niektórzy stosują instrukcje pisane w języku angielskim. Można tak, o ile całość pseudokodu będzie pisana właśnie w tej formie (nie mieszamy, że część instrukcji będzie po polsku, a część po angielsku).

Przykład programu, który nic nie robi, tylko się zaczyna i kończy:

```
START
// Tutaj będzie znajdował się kod programu
STOP
```

Pseudojęzyk

2. Operacje wejścia wyjścia, formatowanie kodu

Opis instrukcji powinien wskazywać na fakt wprowadzenia danych np.:

WCZYTAJ, WPROWADZ, WPISZ, POBIERZ, itp.

Instrukcja stosowana najczęściej do pobierania wartości liczbowych i przechowywania ich jako tzw. zmiennych.

Opis instrukcji powinien wskazywać na fakt wyprowadzenia danych np.:

WYPISZ, DRUKUJ, WYŚWIETŁO, POKAŻ, itp.

Instrukcja stosowana do informowania o wynikach obliczeń i prezentowania dodatkowych informacji.

Możliwe wyświetlenie zarówno zmiennych jak i komunikatów.

Wyświetlenie zmiennej (nazwa zmiennej) powoduje wyświetlenie wartości jaka się znajduje w tej zmiennej.

Wyświetlenie komunikatu (tekst „w cudzysłowie”) powoduje wyświetlenie tekstu w takiej właśnie formie.

Generalnie pseudokod ma prezentować sposób postępowania, nie mówi nic o sposobie pobrania danych i sposobie ich prezentacji na zewnątrz. Dlatego nie ma rygorystycznych reguł jeśli chodzi o opisy instrukcji. Opisy powinny sensownie wskazywać gdzie wprowadzamy, a gdzie wyprowadzamy dane.

Chyba, że zasady stanowią inaczej, np. na 100% wiemy, że wyniki będą drukowane na drukarce, wtedy instrukcja wyprowadzania opisana jako „DRUKUJ” będzie właściwsza niż „WYŚWIETŁO”.

Aby wskazać, że kod należy do „bloku”, do naszego algorytmu, przyjęło się, że stosowane jest wcięcie (wytabowanie) instrukcji znajdujących się pomiędzy początkiem a końcem. Instrukcje wykonują się po kolei (z góry na dół) na tym samym wcięciu, a jeśli ich brakuje, to kolejno na wcięciu „bardziej w lewo”.

Wcięcie powinno być jednoznacznie rozpoznawalne. Pojedyncza spacja to trochę za mało, by czasami je zobaczyć. „Tab” zazwyczaj robi odstęp aż 8 znaków, co wydaje się zbyt wielkim wcięciem. Optymalne wcięcie to 2 lub 4 spacje.

Miedzy różnymi elementami zapisywanymi w jednej linii (np. obliczenia matematyczne) **powinniśmy używać spacji. Zwiększa to znacząco czytelność kodu** (obliczenia „sklejone” ze sobą, zmienne z operatorami, sprawiają, że ciężko zrozumieć o co chodzi w takim wzorze).

Przykład kodu, który tylko drukuje komunikat:

```
START
DRUKUJ "Witaj świecie!"
STOP
```

Pseudojęzyk

3. Operacje na zmiennych (różnych typów)

Użytkownik nie powinien się domyślać co powinien zrobić, powinien to wiedzieć. Znana sytuacja - nie ma nic gorszego niż migający kursor, gdzie nie wiadomo co wpisać, czy liczbę, czy tekst, czy pojedynczy znak.

Dlatego ważne jest informowanie użytkownika co powinien wykonać w danym momencie. Przykładowo instrukcja `Drukuj "Podaj liczbę:"` wskazuje, że należy wprowadzić liczbę (nie znak, nie tekst, nie kolor).

`Wczytaj a` - instrukcja wczytuje wartość i zapisuje ją w zmiennej `a`. Oznacza to, że od tego momentu ilekroć w algorytmie użyjemy zmiennej `a`, będziemy mieli na myśli wartość jaka się w niej znajduje.

Nazwa zmiennej może być praktycznie dowolna. Powinna jednak sugerować swoją nazwą co ona oznacza w programie (nazwanie zmiennej **iloczyn**, kiedy ma przechowywać wynik sumy, nie jest najmądrzejszym pomysłem). **Zmienna nie może być znakiem specjalnym, cyfrą ani liczbą! Nie może zaczynać się od cyfry!** Nie powinna mieć polskich znaków w nazwie. Przyjęło się, że nazwy zmiennej pisane są małymi literami. Jeśli mamy ich mało, a algorytm jest prosty, to w zupełności wystarczy nawet jedna litera np. `a`, `b`, `c`, `x`, `y`, `z`. Jeśli jest ich dużo lub algorytm jest złożony, to zmienne o nazwach np. `suma`, `wynik`, `wieksza`, `mniejsza`, itp. znacznie ułatwiają zrozumienie co jest czym.

Uwaga! Wielkość znaków ma znaczenie! Zmienna `x` to nie jest ta sama zmienna co `x`!

Uwaga! Użycie zmiennej (np. wyświetlenie jej), bez nadania jej wcześniej wartości jest błędem. Dopóki nie podstawimy do zmiennej wartości, taka zmienna nie istnieje, więc nie możemy jej używać!

Uwaga! Komunikaty drukujemy z użyciem cudzysłowu, wartości zmiennych drukujemy bez cudzysłowu.

Jeśli zasady zadania nie mówią inaczej, to w programie można użyć tyle dodatkowych zmiennych pomocniczych, ile tylko chcemy.

Na zmiennych liczbowych możemy wykonywać operacje matematyczne i logiczne. Przyjęło się, że w pseudokodzie **operatorem przypisania** jest `:=` (**operator porównania** `=` jest używany w instrukcjach warunkowych).

Przykład kodu, który wczytuje liczbę, oblicza i drukuje jej kwadrat:

```
START
Drukuj "Podaj liczbę:"
Wczytaj a
kw := a * a
Drukuj "Kwadrat liczby to:"
Drukuj kw
```

STOP

Pseudojęzyk

4. Redukcja instrukcji

Aby zaoszczędzić miejsce i zoptymalizować algorytm stosujemy redukcję bloków. Jeśli mamy kilka bloków tego samego przeznaczenia występujących po sobie, to możemy skrócić taki kod poprzez złączenie elementów w jeden większy. Musimy jednak oddzielić elementy od siebie. Przyjęło się, że używamy do tego celu przecinka.

Takie elementy nie będą wczytane/wyświetlane jednocześnie (w znaczeniu: wszystkie naraz), tylko po kolei, zgodnie z porządkiem ich zapisu.

Nie możemy zredukować kilku bloków w jeden, jeśli mamy mieszaninę bloków wprowadzania i wyprowadzania danych.

Przykładowo kod:

WCZYTAJ a

WCZYTAJ b

WCZYTAJ c

WCZYTAJ d

Realizuje dokładnie takie samo działanie jak zapis:

WCZYTAJ a, b, c, d

Przykładowo kod:

WYPIEJ $"suma: "$

WYPIEJ sm

WYPIEJ $"Różnica: "$

WYPIEJ rz

Realizuje dokładnie takie samo działanie jak zapis:

WYPIEJ $"suma: ", sm, "Różnica: ", rz$

Pseudojęzyk

5. Obliczenia (zmienne liczbowe)

Na zmiennych liczbowych możemy wykonywać operacje matematyczne i logiczne. Przyjęło się, że w pseudokodzie **operatorem przypisania** jest **":="** (**operator porównania** "=" jest używany w instrukcjach warunkowych). $suma := a + b$

Uwaga! To nie jest matematyka!

Wynik obliczeń zawsze jest po lewej stronie operatora ":=".

Możemy także zmieniać wartości zmiennym w trakcie trwania programu. Poniższy zapis nie należy rozumieć w znaczeniu matematycznym. Nie przenosimy takich samych zmiennych na jedną stronę, nie redukujemy wzoru. Najpierw obliczamy wszystko to co jest po prawej stronie operatora przypisania, następnie wynik (to co wyszło) podstawiamy pod zmienną po stronie lewej.

Poniższy przykład pokazuje w jaki sposób zwiększyć wartość zmiennej o

1. $n := n + 1$

Jeśli nie wczytujemy wartości do zmiennych, ani ich nie obliczamy, to musimy ustawić ich wartości początkowe (domyślne). **Nie możemy korzystać ze zmiennych, której wartość jest nieokreślona (nie została ustawiona wcześniej w żaden sposób).**

$zm1 := 0$

$zm2 := zm1 * zm1 + 5$

Obliczenia „dodatkowe” możemy opisać tekstem (aby tylko ich sens był zrozumiały). **Nie używamy zapisu, który nie da się zapisać czcionką drukowaną** (nie zrobimy kreski ułamkowej, nie narysujemy pierwiastka). $pow :=$

$PIERWIASTEK(a)$

$rd := ZOBUJ(1;10)$

... ale nie możemy iść na łatwiznę i stosować zbytich uproszczeń. Jeśli np. musimy wytypować liczbę najmniejszą, to **nie możemy** zapisać: $min :=$

$MINIMUM(a, b, c)$

tylko musimy stworzyć kod, z użyciem instrukcji podstawowych, który takową liczbę wytypuje.

Do potęgi nie używamy operatora „^” (choć wydaje się oczywiste). Wiele języków programowania traktuje operator „^” jako **operator logiczny XOR**. Aby nie mylić operatorów, lepiej zrobić zapis potęgi, np.:

$x := a * a$

$y = POTEGA(a, 2)$

Pseudojęzyk

6. Dzielenie (różne formy)

Dzielenie można realizować z użyciem różnych operatorów. **Każdy z nich działa nieco inaczej, także trzeba wybrać właściwy do sytuacji.**

Dzielenie zwykłe – operator „/”.

Zwraca dokładny wynik operacji. W przykładzie uzyskamy wartość **2.75**.

$a := 11 / 4$

Dzielenie całkowite – operator „div” (divide – podzielny).

Zwraca wynik całkowity z dzielenia. Część „za przecinkiem” (lub reszta) go

nie interesuje i zostaje pominięta. Można używać tylko w sytuacji, gdzie dzielimy liczbę całkowitą przez inną liczbę całkowitą! W przykładzie uzyskamy wartość 2.

$$b := 11 \div 4$$

Reszta z dzielenia – operator „**mod**” (modulo – reszta).

Zwraca resztę z dzielenia. Część całkowita go nie interesuje i zostaje pominięta. Można używać tylko w sytuacji, gdzie dzielimy liczbę całkowitą przez inną liczbę całkowitą! Bardzo przydatne do sprawdzania podzielności liczb! W przykładzie uzyskamy wartość 3.
(bo $11 \bmod 4 = 3$, bo $4 * 2 + 3 = 11$).

$$c := 11 \bmod 4$$

Pseudojęzyk

7. Porównanie z zapisem schematu blokowego

